

56-advanced-ai-dev

Modul 56: Advanced AI Development Workflow

Level	Prerequisites	Output
Advanced	Modul 38 (CI/CD), Modul 53 (AI Code Generation), Modul 54 (AI Testing & Debug)	AI CI/CD pipeline terotomasi penuh dengan code review, testing, security, dan refactoring

Deskripsi

Modul ini membekali peserta dengan kemampuan membangun **AI-driven development pipeline** end-to-end. Dari code review otomatis, testing strategy, security quality gates, hingga refactoring — semua diperkuat oleh agent AI (Claude, GPT, Cline, Copilot). Peserta akan menerapkan AI sebagai **co-pilot** yang aktif di setiap stage siklus pengembangan.

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, peserta mampu:

1. Membangun AI code review pipeline terotomatis berbasis GitHub Actions
2. Menerapkan AI TDD (Red-Green-Refactor dengan bantuan AI)
3. Mengkonfigurasi quality gates dengan security scanning otomatis
4. Melakukan AI-assisted refactoring kode monolitik ke arsitektur modern



Figure 1: Banner

Tools & Teknologi

- **GitHub Actions** — workflow automation
- **OpenAI / Claude API** — AI review & code generation
- **Cline / Claude Code** — terminal AI agent
- **Codacy / SonarCloud** — static analysis & code quality
- **ESLint / Prettier / TypeScript** — linting & type checking
- **OWASP ZAP / Snyk / Trivy** — security scanning
- **Vitest / Jest / Playwright** — testing frameworks
- **GitHub Secrets / Doppler** — secret management

Sesi

#	Judul	Durasi
1	AI Code Review Pipeline	3 jam
2	AI Testing Strategy	3 jam
3	AI Security & Quality Gates	3 jam
4	AI Refactoring & Migration	3 jam

Struktur Repo

```
56-advanced-ai-dev/  
├── README.md  
├── 01-ai-code-review-pipeline.md  
├── 02-ai-testing-strategy.md  
├── 03-ai-security-quality.md  
└── 04-ai-refactoring.md
```

Referensi

- [GitHub Actions Documentation](#)
- [OpenAI API — Code Review](#)
- [OWASP Top 10](#)
- [SonarCloud Quality Gates](#)
- [Cline — AI Terminal Agent](#)

Sesi 1: AI Code Review Pipeline

Durasi: 3 jam

Tujuan

Setelah sesi ini, peserta mampu: - Memahami arsitektur AI code review pipeline - Mengkonfigurasi GitHub Actions + AI agent untuk review otomatis - Menentukan area review, severity level, dan auto-fix commit - Mengintegrasikan tools seperti Cline, Codacy, SonarCloud

1. Konsep AI Code Review

1.1 Apa Itu AI Code Review?

AI code review adalah proses dimana agent AI (Claude, GPT, Cline) secara otomatis mereview pull request berdasarkan aturan yang telah ditentukan. Review mencakup analisis kode, saran perbaikan, hingga auto-commit fix.

1.2 Trigger Pipeline

Pipeline dipicu oleh event `pull_request` atau `pull_request_target`:

```
on:  
  pull_request:  
    types: [opened, synchronize, reopened]
```

1.3 Area Review

Area	Deskripsi	Contoh
Code Style	Konsistensi format, naming convention, imports	snake_case vs camelCase
Logic Bugs	Race condition, off-by-one, null pointer	Loop boundary error
Security Vuln	Injection, XSS, hardcoded secret	SQLi risk in raw query
Performance	N+1 query, memory leak, unnecessary re-render	Missing index
Test Coverage	Missing test, low assertion quality	Function without tests
API Design	RESTful compliance, breaking changes, versioning	Missing validation

1.4 Severity Level

CRITICAL – blocking, must fix before merge
MAJOR – should fix, high impact
MINOR – nice to fix, low impact
SUGGESTION – informational, optional

2. Pipeline Arsitektur

[PR Created] → [Checkout Code] → [AI Review Agent] → [Post Comment] → [Auto-fix Branch] → [Approve/Request Changes]

2.1 Workflow GitHub Actions

```
# .github/workflows/ai-code-review.yml
name: AI Code Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: AI Review via OpenAI
        uses: openai/code-review-action@v1
        with:
          openai-api-key: ${ secrets.OPENAI_API_KEY }
          github-token: ${ secrets.GITHUB_TOKEN }
          model: gpt-4o
          review-areas: style,security,performance,api-design
          severity-threshold: major
          auto-fix: false

      - name: Run Static Analysis
        uses: codacy/codacy-analysis-cli-action@v4

      - name: Run Linter
        run: |
```

```
npm run lint
npm run format:check
```

2.2 Integration dengan Cline (Terminal AI Agent)

Cline bisa dijalankan dalam pipeline untuk review mendalam:

```
- name: Cline Review
  run: |
    cline review \
      --diff origin/main...HEAD \
      --format markdown \
      --output review.md
    cat review.md >> $GITHUB_STEP_SUMMARY
```

2.3 Integration dengan SonarCloud

```
- name: SonarCloud Scan
  uses: SonarSource/sonarcloud-github-action@v2
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

3. Prompt AI untuk Code Review

3.1 System Prompt

You are a senior code reviewer. Analyze the diff below and provide feedback. Focus on: code style, logic bugs, security vulnerabilities, performance issues, test coverage gaps, and API design violations.

Use this format for each finding:

```
[SEVERITY: critical|major|minor|suggestion]
**File**: `path/to/file.ts`
**Line**: L12-L18
**Issue**: Description
**Suggestion**: How to fix
**Code**:
```suggestion
// fixed code here
```

Sort findings by severity (critical first). If auto-fix enabled, include a git commit message suggestion.

### ### 3.2 Review Comment Format

Setiap finding dikirim sebagai PR comment:

#### □ **AI Code Review**

##### **Critical (2)**

- **L12-L18** — SQL injection risk in userQuery
- **L45** — Hardcoded API key detected

##### **Major (3)**

- **L67-L72** — N+1 query in user list endpoint
- **L89** — Missing input validation

##### **Minor (1)**

- **L102** — Unused import lodash

##### **Suggestion (4)**

- **L120** — Consider using optional chaining

---

## ## 4. Auto-Fix

### ### 4.1 Suggested Fix Commit

AI dapat membuat commit fix langsung ke branch PR:

```
```yaml
- name: Apply AI Fixes
  if: github.event_name == 'pull_request'
  run: |
    git config user.name "ai-bot"
    git config user.email "ai-bot@example.com"
    # Apply patch files from AI review
    for patch in .ai-review/fixes/*.patch; do
      git apply "$patch"
    done
    git add -A
    git commit -m "fix: AI auto-fix applied"
    git push
```

4.2 Conditional Auto-Approval

```
- name: Auto Approve if Critical Issues Zero
  if: steps.review.outputs.critical_count == '0'
  uses: hmarr/auto-approve-action@v4
  with:
    github-token: ${ secrets.GITHUB_TOKEN }
```

5. Review Areas Detail

5.1 Code Style Check

```
// ❌ Bad
const x = function(y){return y+1}
// ✅ Good
const increment = (num: number): number => num + 1;
```

5.2 Logic Bug Detection

```
// ❌ Off-by-one – loop skips last element
for (let i = 0; i < arr.length - 1; i++) { ... }
// ✅ Fix
for (let i = 0; i < arr.length; i++) { ... }
```

5.3 Security Vulnerability

```
// ❌ SQL Injection
const query = `SELECT * FROM users WHERE id = ${userId}`;
// ✅ Parameterized query
const query = 'SELECT * FROM users WHERE id = ?';
db.execute(query, [userId]);
```

5.4 Performance Anti-pattern

```
// ❌ N+1 – query per item
const items = await Item.findAll();
for (const item of items) {
  const user = await User.findByPk(item.userId);
}
// ✅ Eager loading
const items = await Item.findAll({ include: [User] });
```


5.5 API Design Violation

```
// ☐ Breaking change – removing field
app.put('/api/v1/users/:id', (req, res) => {
  delete req.body.username; // breaks clients
});
// ☐ Deprecate first, remove later
```

6. Tools & Integrasi

6.1 Codacy

```
- name: Codacy Analysis
  uses: codacy/codacy-analysis-cli-action@v4
  with:
    project-token: ${ secrets.CODACY_PROJECT_TOKEN }
    max-allowed-issues: 50
```

6.2 SonarCloud

```
- name: SonarCloud
  uses: SonarSource/sonarcloud-github-action@v2
  with:
    args: >
      -Dsonar.qualitygate.wait=true
      -Dsonar.qualitygate.timeout=300
```

6.3 ESLint + Prettier

```
- name: Lint & Format Check
  run: |
    npx eslint . --max-warnings=0
    npx prettier --check .
```

6.4 Trivy (Security Scan)

```
- name: Trivy Scan
  uses: aquasecurity/trivy-action@master
  with:
    scan-type: fs
    scan-ref: .
    severity: HIGH,CRITICAL
    exit-code: 1
```

7. Latihan: Setup AI Code Review Pipeline

Tugas

1. Buat repository GitHub dengan project Node.js sederhana (Express API)
2. Setup **AI Code Review** workflow menggunakan OpenAI/Claude
3. Konfigurasi **area review**: security, performance, API design
4. Integrasikan **SonarCloud** untuk code quality scan
5. Tambahkan **auto-fix commit** untuk issue severity minor
6. Buat PR dengan kode yang sengaja mengandung bug untuk di-review

Kriteria Sukses

- ☐ Workflow `.github/workflows/ai-code-review.yml` berfungsi
- ☐ AI memberikan komentar review di setiap PR
- ☐ Auto-fix berhasil membuat commit perbaikan
- ☐ SonarCloud melaporkan quality gate
- ☐ Setidaknya 1 issue critical, 2 major, 3 minor terdeteksi

Starter Template

```
// src/server.js – sengaja mengandung bug
const express = require('express');
const app = express();

// Security: SQL Injection
app.get('/user/:id', (req, res) => {
  const query = `SELECT * FROM users WHERE id = ${req.params.id}`;
  db.query(query, (err, result) => {
    res.json(result);
  });
});

// Performance: N+1
app.get('/orders', async (req, res) => {
  const users = await User.findAll();
  const orders = [];
  for (const user of users) {
    const userOrders = await Order.findAll({ where: { userId: user.id } });
    orders.push(...userOrders);
  }
  res.json(orders);
});
```

```
// API Design: No validation
app.post('/user', (req, res) => {
  User.create(req.body); // no sanitization
  res.status(201).send('OK');
});

app.listen(3000);
```

Checklist Review

Item	Sudah
Workflow terdeteksi AI review	☐
Komentar review muncul di PR	☐
Issue severity terklasifikasi	☐
Auto-fix commit terbuat	☐
SonarCloud quality gate aktif	☐
Semua bug terdeteksi	☐

Referensi

- [OpenAI Code Review Action](#)
- [GitHub Actions — Events that trigger workflows](#)
- [Codacy CLI](#)
- [SonarCloud Quality Gates](#)
- [Cline Documentation](#)

Sesi 2: AI Testing Strategy

Durasi: 3 jam

Tujuan

Setelah sesi ini, peserta mampu:

- Menerapkan AI TDD loop (Red → Green → Refactor)
- Mengenerate unit test, integration test, dan E2E test dengan AI
- Memanfaatkan AI untuk edge case detection & fuzzing
- Mengelola test maintenance otomatis saat kode berubah

1. AI TDD (Test-Driven Development with AI)

1.1 Siklus AI TDD

RED Write test prompt → AI generate test suite Run → fail (red)
GREEN AI generate implementation to pass tests Run → pass (green)
REFACTOR AI refactor code & update tests Run → all green (refactored)

1.2 Contoh AI TDD Loop

Step 1 — RED: Prompt AI untuk generate test

You are a test engineer. Write unit tests for a function `calculateDiscount(price, userType)`. Requirements:

- Regular user: no discount
 - Premium user: 20% discount
 - VIP user: 50% discount with max cap \$100
 - Edge cases: negative price, zero price, null userType
- Use Vitest (TypeScript).

AI Output — Test Suite:

```
// calculateDiscount.test.ts
import { describe, it, expect } from 'vitest';
import { calculateDiscount } from './calculateDiscount';

describe('calculateDiscount', () => {
  // Happy path
  it('returns full price for regular user', () => {
    expect(calculateDiscount(100, 'regular')).toBe(100);
  });

  it('applies 20% discount for premium user', () => {
    expect(calculateDiscount(100, 'premium')).toBe(80);
  });

  it('applies 50% discount for VIP user with max $100 cap', () => {
    expect(calculateDiscount(100, 'vip')).toBe(50);
  });
});
```

```

    expect(calculateDiscount(500, 'vip')).toBe(400); // 500 - 100 = 400
  });

  // Edge cases
  it('returns 0 for negative price', () => {
    expect(calculateDiscount(-10, 'regular')).toBe(0);
  });

  it('returns 0 for zero price', () => {
    expect(calculateDiscount(0, 'premium')).toBe(0);
  });

  it('throws for null userType', () => {
    expect(() => calculateDiscount(100, null)).toThrow();
  });

  it('throws for unknown userType', () => {
    expect(() => calculateDiscount(100, 'unknown')).toThrow();
  });
});

```

Step 2 — GREEN: Prompt AI untuk generate implementation

Implement calculateDiscount function to pass all tests above.

AI Output — Implementation:

```

// calculateDiscount.ts
export function calculateDiscount(price: number, userType: string | null): number {
  if (price <= 0) return 0;
  if (!userType) throw new Error('userType is required');

  const discounts: Record<string, number> = {
    regular: 0,
    premium: 0.2,
    vip: 0.5,
  };

  if (!(userType in discounts)) {
    throw new Error(`Unknown userType: ${userType}`);
  }

  const discount = price * discounts[userType];

  // VIP cap
  const maxDiscount = userType === 'vip' ? 100 : Infinity;
  const appliedDiscount = Math.min(discount, maxDiscount);
}

```

```
    return price - appliedDiscount;
}
```

Step 3 — REFACTOR: Prompt AI

Refactor calculateDiscount – extract discount config, improve readability. Update tests if needed. All tests must still pass.

2. Test Generation

2.1 Unit Test Generation

Prompt:

Generate unit tests for this function. Cover:

- Normal cases (happy path)
- Boundary values
- Error states
- Null/undefined inputs

```
function paginate<T>(items: T[], page: number, pageSize: number): { data: T[]; total: number } {
    const start = (page - 1) * pageSize;
    const data = items.slice(start, start + pageSize);
    return {
        data,
        total: items.length,
        page,
        totalPages: Math.ceil(items.length / pageSize),
    };
}
```

AI-Generated Tests:

```
describe('paginate', () => {
    const items = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

    it('returns first page', () => {
        const result = paginate(items, 1, 5);
        expect(result.data).toEqual([1, 2, 3, 4, 5]);
        expect(result.totalPages).toBe(2);
    });

    it('returns second page', () => {
        const result = paginate(items, 2, 5);
        expect(result.data).toEqual([6, 7, 8, 9, 10]);
    });
});
```

```

it('handles page beyond total', () => {
  const result = paginate(items, 10, 5);
  expect(result.data).toEqual([]);
  expect(result.page).toBe(10);
});

it('handles pageSize larger than total', () => {
  const result = paginate(items, 1, 100);
  expect(result.data).toEqual(items);
  expect(result.totalPages).toBe(1);
});

it('handles empty array', () => {
  const result = paginate([], 1, 5);
  expect(result.data).toEqual([]);
  expect(result.total).toBe(0);
  expect(result.totalPages).toBe(0);
});

it('handles page 0 gracefully', () => {
  const result = paginate(items, 0, 5);
  expect(result.data).toEqual([1, 2, 3, 4, 5]); // start = -5, slice handles i
});
});

```

2.2 Integration Test Generation

Prompt:

Generate integration tests for this Express endpoint:

POST /api/users

Body: { name: string, email: string, role: 'admin' | 'user' }

Response: 201 { id, name, email, role, createdAt }

Errors: 400 if missing fields, 409 if email exists

Use Supertest + Vitest. Mock database calls.

2.3 E2E Test Generation

Prompt:

Generate Playwright E2E tests for:

1. User registration flow
2. Login → dashboard redirect
3. Form validation errors display

4. API error toast notification

AI Output Example:

```
import { test, expect } from '@playwright/test';

test.describe('User Registration', () => {
  test('successful registration redirects to dashboard', async ({ page }) => {
    await page.goto('/register');
    await page.fill('[data-testid="name"]', 'Test User');
    await page.fill('[data-testid="email"]', 'test@example.com');
    await page.fill('[data-testid="password"]', 'SecurePass123!');
    await page.click('[data-testid="submit"]');
    await expect(page).toHaveURL('/dashboard');
    await expect(page.locator('[data-testid="welcome"]')).toContainText('Test User');
  });

  test('shows validation errors on empty form', async ({ page }) => {
    await page.goto('/register');
    await page.click('[data-testid="submit"]');
    await expect(page.locator('[data-testid="field-error"]')).toHaveCount(3);
  });

  test('shows API error toast on duplicate email', async ({ page }) => {
    await page.goto('/register');
    await page.fill('[data-testid="name"]', 'Existing User');
    await page.fill('[data-testid="email"]', 'existing@example.com');
    await page.fill('[data-testid="password"]', 'SecurePass123!');
    await page.click('[data-testid="submit"]');
    await expect(page.locator('[data-testid="toast-error"]')).toContainText('Email already exists');
  });
});
```

3. Edge Cases & Fuzzing

3.1 AI-Powered Edge Case Detection

Prompt:

Analyze this function and list ALL possible edge cases I might miss.
Provide test cases for each.

```
function processPayment(amount: number, currency: string, method: string) {
  // ... complex logic
}
```


AI Output — Edge Cases Found:

Edge Case	Why It Matters	Test
Amount in cents vs decimal	100 vs 1.00 treat differently	<code>processPayment(0.01, 'USD', 'card')</code>
Currency with no decimals (JPY)	JPY ¥100 = integer, not 1.00	<code>processPayment(100, 'JPY', 'card')</code>
Unsupported currency	Should throw, not silently fail	<code>processPayment(50, 'XYZ', 'card')</code>
Method null	NPE if not checked	<code>processPayment(50, 'USD', null)</code>
Amount = 0	Free transaction allowed?	<code>processPayment(0, 'USD', 'card')</code>
Floating point precision	$0.1 + 0.2 \neq 0.3$	<code>processPayment(0.1, 'USD', 'card')</code>

3.2 Property-Based Testing

AI generate property tests menggunakan fast-check:

```
import fc from 'fast-check';

describe('paginate property-based tests', () => {
  it('total items in all pages equals original length', () => {
    fc.assert(
      fc.property(
        fc.array(fc.anything()),
        fc.integer({ min: 1, max: 10 }),
        fc.integer({ min: 1, max: 10 }),
        (items, page, pageSize) => {
          const result = paginate(items, page, pageSize);
          const allItems = result.data; // just one page
          return true; // invariant: sum of items across all pages = total
        }
      )
    );
  });
});
```

3.3 Fuzzing

AI-generated fuzz input

```
npm run fuzz -- --provider random --iterations 10000
```

Prompt fuzzing:

Generate 100 random valid and invalid JSON payloads for POST /api/users.

Include: missing fields, wrong types, XSS payloads, SQL injection, huge payloads, Un

Output as JSON array.

4. Test Maintenance

4.1 AI Update Tests When Code Changes

Scenario: Function signature berubah — parameter userType diganti role.

The function calculateDiscount changed:

- OLD: calculateDiscount(price: number, userType: string)
- NEW: calculateDiscount(price: number, role: 'customer' | 'member' | 'vip')
- Customer: 0% discount
- Member: 15% discount
- VIP: 40% discount (no cap anymore)

Update all existing tests to match new signature and behavior.

4.2 Detect and Fix Flaky Tests

Analyze this test output. It fails intermittently:

- PASS: 47/50 runs
- FAIL: 3/50 runs with "Element not found: #submit-btn"

Possible causes:

- a) Race condition — button not rendered yet
- b) Network latency — API response slow
- c) Async not awaited

Fix the test with proper waits.

4.3 Coverage Gap Detection

Analyze test coverage report:

- Lines: 78%
- Branches: 62%
- Functions: 85%

Generate tests for uncovered branches in:
src/services/payment.ts (branches: 45%)
src/utils/validators.ts (lines: 55%)

5. AI Test Suite Generator

5.1 Prompt Template Lengkap

Generate a complete test suite for [component/function].

Testing framework: Vitest
Coverage threshold: 80% lines, 75% branches
Include: unit, integration (if applicable)
Edge cases: null, undefined, empty, boundary, invalid types
Mock: external API calls, database queries
Format: TypeScript

Code to test:

[PASTE CODE]

5.2 Example: Generate for API Route

```
// AI-Generated: tests/api/users.test.ts
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { createUser } from '../api/users';
import { db } from '../lib/db';
import { sendWelcomeEmail } from '../lib/email';

vi.mock('../lib/db');
vi.mock('../lib/email');

describe('POST /api/users', () => {
  beforeEach(() => {
    vi.clearAllMocks();
  });

  it('creates user and sends welcome email', async () => {
    vi.mocked(db.query).mockResolvedValue({ rows: [] });

    const result = await createUser({
      name: 'Test',
      email: 'test@test.com',
    });
  });
});
```

```

    });

    expect(db.query).toHaveBeenCalledWith(
      'INSERT INTO users (name, email) VALUES ($1, $2) RETURNING *',
      ['Test', 'test@test.com']
    );
    expect(sendWelcomeEmail).toHaveBeenCalledWith('test@test.com');
    expect(result.status).toBe(201);
  });

  it('returns 409 on duplicate email', async () => {
    vi.mocked(db.query).mockRejectedValue({ code: '23505' }); // unique violation

    await expect(
      createUser({ name: 'Test', email: 'dup@test.com' })
    ).rejects.toMatchObject({ status: 409 });
  });
});

```

6. Latihan: AI TDD Loop Lengkap

Tugas

1. Pilih satu fungsi kompleks (validasi email, parser CSV, auth middleware)
2. **RED** — Minta AI generate test suite untuk fungsi tsb
3. Simpan test, jalankan — pastikan fail
4. **GREEN** — Minta AI generate implementasi passing semua test
5. Simpan implementasi, jalankan test — pastikan pass
6. **REFACTOR** — Minta AI refactor kode, update test jika perlu
7. Generate **property-based test** untuk fungsi tsb
8. Buat **fuzz test** dengan 100 input random

Starter

You are my TDD partner. I want to build a function: `validatePassword(password: string)`

Rules:

- Min 8 chars, max 64 chars
- At least 1 uppercase, 1 lowercase, 1 digit, 1 special char
- No spaces allowed
- Must not contain common patterns: "password", "12345", "qwerty"

Generate the test suite first (Vitest, TypeScript).

Kriteria Sukses

- ☐ AI TDD loop berjalan minimal 2 iterasi
- ☐ Test suite mencakup >90% edge cases
- ☐ Property-based test menemukan minimal 1 bug
- ☐ Fuzz test tidak crash unhandled
- ☐ Test maintenance: ketika kode berubah, AI update test otomatis

Catatan

- Gunakan claude atau gpt via terminal (Cline/Codex)
 - Atau gunakan API langsung via script
 - Simpan semua prompt dan output sebagai referensi
-

Referensi

- [Vitest Documentation](#)
- [Playwright Test Generator](#)
- [fast-check — Property Based Testing](#)
- [OWASP Fuzzing Guide](#)
- [WWCode — AI TDD](#)

Sesi 3: AI Security & Quality Gates

Durasi: 3 jam

Tujuan

Setelah sesi ini, peserta mampu: - Melakukan AI-assisted security audit (OWASP Top 10, dependency check, SAST) - Mengkonfigurasi quality gates pipeline (lint → type-check → test → coverage → security → build) - Menerapkan automated gating — PR diblokir jika kualitas di bawah threshold - Menjalankan secret detection otomatis

1. AI Security Audit

1.1 Arsitektur Security Pipeline

[Push/PR] → [SAST Scan] → [Dependency Check] → [Secret Detection] → [DAST] → [Report]

↑ ↑ ↑ ↑ ↑

ESLint Snyc/ TruffleHog/ OWASP ZAP AI Audit

TypeScript (AST-based)	Trivy/ Dependabot	Gitleaks	Summary
---------------------------	----------------------	----------	---------

1.2 OWASP Top 10 Scanning dengan AI

Prompt AI untuk security audit:

You are a security engineer. Perform an OWASP Top 10 (2021) audit on this code. For each vulnerability found, provide:

1. OWASP category (A01-A10)
2. Location (file:line)
3. Risk level (Critical/High/Medium/Low)
4. Remediation code
5. CWE reference if applicable

Code:
[PASTE CODE]

Contoh Output AI Audit:

OWASP	Vulnerability	Location	Risk
A01 — Broken Access Control	No auth check on /admin/*	routes/admin.js:20	Critical
A03 — Injection	SQL query concatenation	db/users.js:45	Critical
A05 — Security Misconfigu- ration	CORS * in production	app.js:8	High
A06 — Vulnerable Compo- nents	Express v4.16.0 (known RCE)	package.json	High
A09 — Logging Failures	Passwords logged in plaintext	auth.js:23	Medium

1.3 SAST (Static Application Security Testing)

```
# .github/workflows/security-scan.yml
name: Security Scan
on: [pull_request]
```

```

jobs:
  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: CodeQL Analysis
        uses: github/codeql-action/analyze@v3
        with:
          languages: javascript, typescript
          queries: security-extended, security-and-quality

      - name: ESLint Security Plugin
        run: |
          npx eslint . --config .eslintrc-security.js
          # .eslintrc-security.js extends 'plugin:security/recommended'

      - name: AI Security Review
        uses: openai/code-review-action@v1
        with:
          openai-api-key: ${ secrets.OPENAI_API_KEY }
          review-areas: security
          severity-threshold: high

```

1.4 Dependency Check

```

- name: Snyk Dependency Scan
  uses: snyk/actions/node@master
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
  with:
    args: --severity-threshold=high

- name: Trivy Filesystem Scan
  uses: aquasecurity/trivy-action@master
  with:
    scan-type: fs
    scan-ref: .
    format: sarif
    output: trivy-results.sarif
    exit-code: 1
    severity: HIGH,CRITICAL

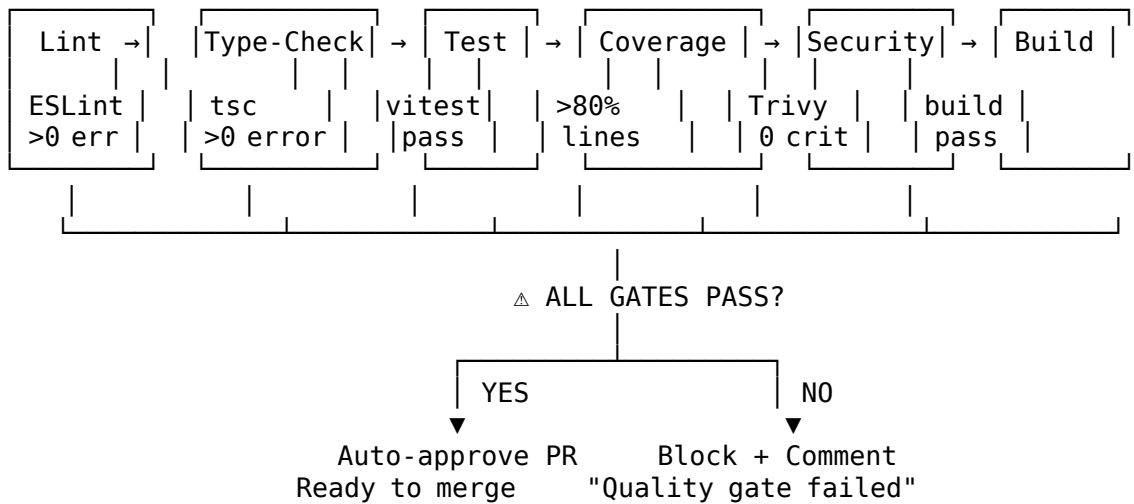
- name: Dependabot Alerts (GitHub-native)
  # Enabled via repository Settings → Security → Dependabot

```

Automatically creates PRs for vulnerable deps

2. Quality Gates Pipeline

2.1 Pipeline Stage Definition



2.2 Quality Gate Configuration

```
# .github/workflows/quality-gates.yml
name: Quality Gates
on:
  pull_request:
    types: [opened, synchronize]
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Lint
        id: lint
        run: |
          npm run lint
        continue-on-error: true

      - name: TypeScript Check
```



```

    id: typecheck
    run: |
        npx tsc --noEmit
    continue-on-error: true

- name: Test
  id: test
  run: |
      npm run test -- --coverage
  continue-on-error: true

- name: Coverage
  id: coverage
  run: |
      npx istanbul check-coverage --statements 80 --branches 70 --functions
  continue-on-error: true

- name: Security Scan
  id: security
  run: |
      npx trivy fs . --exit-code 1 --severity CRITICAL,HIGH
  continue-on-error: true

- name: Build
  id: build
  run: |
      npm run build
  continue-on-error: true

- name: Gate Check
  if: always()
  run: |
      FAILED=""
      ${ steps.lint.outcome == 'failure' && 'FAILED+= "Lint "' }
      ${ steps.typecheck.outcome == 'failure' && 'FAILED+= "TypeScript "' }
      ${ steps.test.outcome == 'failure' && 'FAILED+= "Test "' }
      ${ steps.coverage.outcome == 'failure' && 'FAILED+= "Coverage "' }
      ${ steps.security.outcome == 'failure' && 'FAILED+= "Security "' }
      ${ steps.build.outcome == 'failure' && 'FAILED+= "Build "' }

      if [ -n "$FAILED" ]; then
          echo "## ☐ Quality Gates Failed" >> $GITHUB_STEP_SUMMARY
          echo "$FAILED" >> $GITHUB_STEP_SUMMARY
          exit 1
      else
          echo "## ☑ All Quality Gates Passed" >> $GITHUB_STEP_SUMMARY

```

fi

2.3 Threshold Matrix

Gate	Threshold	Action on Fail
Lint	Zero errors, warnings < threshold	PR labelled blocked:lint
TypeScript	Zero tsc --noEmit errors	PR labelled blocked:types
Unit Test	All passing	PR labelled blocked:tests
Coverage	Lines ≥80%, Branches ≥70%	PR labelled blocked:coverage
Security	Zero CRITICAL/HIGH vulnerabilities	PR labelled blocked:security
Build	Zero build errors	PR labelled blocked:build

3. Automated Gating

3.1 PR Blocked if Below Threshold

```
name: PR Gate
on:
  pull_request:
    types: [opened, synchronize, labeled, unlabeled]

jobs:
  check-gates:
    runs-on: ubuntu-latest
    outputs:
      status: ${ steps.gate.outputs.status }
    steps:
      - id: gate
        run: |
          # Check all gate statuses from previous workflow
          if [ "${ needs.quality.result }" == "success" ]; then
            echo "status=passed" >> $GITHUB_OUTPUT
          else
            echo "status=blocked" >> $GITHUB_OUTPUT
          fi

  block-pr:
    needs: check-gates
```

```

if: needs.check-gates.outputs.status == 'blocked'
runs-on: ubuntu-latest
steps:
  - uses: actions/github-script@v7
    with:
      script: |
        github.rest.issues.createComment({
          owner: context.repo.owner,
          repo: context.repo.repo,
          issue_number: context.issue.number,
          body: `## ❌ Quality Gate Blocked\n\nThis PR cannot be merged until
        });

```

3.2 Branch Protection Rule

Set branch protection rules yang membutuhkan status check:

Settings → Branches → Add rule:

- Require status checks to pass before merging
 - Require branches to be up-to-date
 - Status checks: "Quality Gates / quality" must pass
 - Include administrators
-

4. Secret Detection

4.1 AI Scan untuk Hardcoded Secrets

Prompt AI:

Scan this repository for hardcoded secrets:

- API keys (sk-..., AKIA..., etc.)
- Database URLs with credentials
- Private keys (-----BEGIN...)
- OAuth tokens
- Passwords
- .env files committed

Report each finding with:

- File path and line
- Secret type
- Risk level
- Remediation: use GitHub Secrets / Doppler / Vault

4.2 Automated Secret Detection Pipeline

```
# .github/workflows/secret-detection.yml
name: Secret Detection
on: [push, pull_request]

jobs:
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Gitleaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

      - name: TruffleHog
        run: |
          trufflehog filesystem --directory=. --json | tee trufflehog-results.json
          if [ -s trufflehog-results.json ]; then
            echo "🔍 Secrets detected!" >> $GITHUB_STEP_SUMMARY
            jq -r '. | "\(.SourceMetadata.Data.Filesystem.file):\(.SourceMetadata.Data.Filesystem.file)'" >> $GITHUB_STEP_SUMMARY
            exit 1
          fi

      - name: Check for .env committed
        run: |
          if git ls-files | grep -q '\.env$'; then
            echo "🔍 .env file committed to repo!" >> $GITHUB_STEP_SUMMARY
            exit 1
          fi

      - name: AI Secret Review
        run: |
          cline review --target secrets --json > ai-secret-report.json
```

4.3 Secret Detection Ruleset

```
{
  "rules": [
    { "pattern": "sk-[a-zA-Z0-9]{20,}", "type": "OpenAI API Key", "severity": "critical" },
    { "pattern": "AKIA[0-9A-Z]{16}", "type": "AWS Access Key", "severity": "critical" },
    { "pattern": "-----BEGIN (RSA|EC|OPENSSH) PRIVATE KEY-----", "type": "Private Key", "severity": "critical" }
```

```

    { "pattern": "password\\s*[=:]\\s*['\"](?:[^\"]|\"(?:\"|\\\\)\")*\"", "type": "Password", "severity": "critical", "category": "Credentials" },
    { "pattern": "ghp_[a-zA-Z0-9]{36}", "type": "GitHub Token", "severity": "critical", "category": "Credentials" },
    { "pattern": "postgres://\\w+:\\w+@", "type": "DB URL with Credentials", "severity": "critical", "category": "Credentials" }
  ]
}

```

5. AI Security Remediation

5.1 Auto-Fix Security Issues

Prompt:

Generate a fix for this security vulnerability:

Vulnerability: SQL Injection in login endpoint

File: src/routes/auth.js:15-22

Code:

```
const query = `SELECT * FROM users WHERE email = '${email}' AND password = '${password}'`
```

Requirements:

1. Use parameterized query
2. Hash password comparison (bcrypt)
3. Rate limiting on failed attempts
4. Return generic error (don't reveal which field is wrong)

Output: Full fixed code with explanation.

5.2 AI Remediation Workflow

```

- name: AI Security Fix
  if: steps.security.outcome == 'failure'
  run: |
    # AI generates fixes for detected issues
    cline fix --from trivy-results.sarif --output .ai-fixes/
    git apply .ai-fixes/*.patch
    git commit -m "fix: AI security remediation"
    git push

```

6. Latihan: Setup Quality Gates Pipeline

Tugas

1. Buat GitHub Actions workflow quality-gates.yml

2. Konfigurasi 6 stage: lint → type-check → test → coverage → security → build
3. Set threshold: coverage $\geq 80\%$, security zero critical, lint zero errors
4. Tambahkan **secret detection** dengan Gitleaks atau TruffleHog
5. Buat **branch protection rule** yang require status check
6. Buat PR dengan kode yang mengandung:
 - Hardcoded API key
 - SQL injection
 - Coverage di bawah threshold
 - Linting error

Kriteria Sukses

- ☐ PR otomatis diblokir saat quality gate gagal
- ☐ Secret terdeteksi dan menampilkan warning
- ☐ Coveragedi bawah 80% menyebabkan gate fail
- ☐ Linting error muncul di PR comment
- ☐ Security vulnerability (SQLi) terdeteksi
- ☐ Setelah semua gate pass, PR bisa di-merge

Starter Template

```
# Starter quality-gates.yml – isi threshold yang sesuai
name: Quality Gates
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup
        run: npm ci

      - name: Lint
        run: npm run lint
        continue-on-error: true

      - name: Test with Coverage
        run: npm run test -- --coverage
```

- name: Security Scan
run: npx trivy fs . --severity CRITICAL,HIGH
continue-on-error: true
- name: Secret Detection
run: npx gitleaks detect --source . --verbose
- name: Build
run: npm run build
continue-on-error: true
- name: Gate Verdict
if: always()
run: |
TODO: implement gate logic
echo "Gate status: pending implementation"

Checklist Audit

Item	Status
OWASP Top 10 scan berjalan	☐
SAST (CodeQL/ESLint security) aktif	☐
Dependency check (Snyk/Trivy) aktif	☐
Secret detection aktif	☐
Branch protection rule terkonfigurasi	☐
PR otomatis blocked saat gate fail	☐
AI security report di PR comment	☐

Referensi

- [OWASP Top 10 \(2021\)](#)
- [GitHub CodeQL](#)
- [Gitleaks](#)
- [TruffleHog](#)
- [Snyk Documentation](#)
- [Trivy](#)
- [GitHub Branch Protection Rules](#)

Sesi 4: AI Refactoring & Migration

Durasi: 3 jam

Tujuan

Setelah sesi ini, peserta mampu: - Mendeteksi code smell dengan AI (long function, large class, duplicated code) - Menyusun dan menjalankan strategi refactoring (plan → execute → verify → commit) - Melakukan migration patterns: library upgrade, framework migration, pattern change - Mengekstrak service dari monolith dengan AI assistance - Menangani database migration dengan rollback plan

1. AI Code Smell Detection

1.1 Jenis Code Smell

Smell	Deteksi AI	Severity
Long Function	Function >50 lines	Medium
Large Class	Class >500 lines, >20 methods	High
Duplicated Code	Similar blocks >10 lines across files	High
Comment-as-Doc	Comments explaining what, not why	Low
God Object	Single class knows/does too much	Critical
Shotgun Surgery	One change requires edits in many files	High
Feature Envy	Method more interested in other class data	Medium
Primitive Obsession	Using primitives instead of value objects	Low
Long Parameter List	>4 parameters in function	Medium
Dead Code	Unused functions, variables, imports	Low

1.2 AI Detection Prompt

Analyze this codebase for code smells. For each finding:

1. Smell type (from list above)
2. File:line
3. Severity (High/Medium/Low)
4. Why it's a problem
5. Refactoring suggestion

List smells sorted by severity.

[PASTE CODE or REPO PATH]

1.3 Example: AI Detects Long Function

```
// □ BEFORE – Long function (72 lines)
async function processOrder(orderId, userId, paymentMethod, shippingAddress, couponCode) {
  // validation
  if (!orderId) throw new Error('Missing orderId');
  if (!userId) throw new Error('Missing userId');
  if (!paymentMethod) throw new Error('Missing payment method');
  // ... 10 more validation lines

  // fetch data
  const order = await Order.findById(orderId);
  const user = await User.findById(userId);
  const product = await Product.findById(order.productId);
  const inventory = await Inventory.findByProductId(product.id);
  // ... more fetching

  // calculate pricing
  let subtotal = product.price * order.quantity;
  let discount = 0;
  if (couponCode) {
    const coupon = await Coupon.findByCode(couponCode);
    discount = subtotal * (coupon.discountPercent / 100);
  }
  let tax = (subtotal - discount) * 0.1;
  let shipping = shippingMethod === 'express' ? 15 : 5;
  let total = subtotal - discount + tax + shipping;
  // ... 30 more lines
}
```

AI Suggestion:

Smell: Long Function (72 lines) – SEVERITY: HIGH

File: src/services/orderService.js:1

Problem: Does validation, data fetching, pricing, inventory check, payment, and notification in one function. Violates Single Responsibility.

Fix: Extract into methods:

- validateInputs(orderId, userId, paymentMethod)
- fetchOrderData(orderId, userId, productId)
- calculatePricing(product, quantity, couponCode)
- processPayment(order, paymentMethod)

```
- sendNotifications(user, order)
```

1.4 Example: AI Detects Duplicated Code

```
// □ BEFORE – Duplicated validation logic
// File 1: userController.js
function validateEmail(email) {
  const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!re.test(email)) throw new Error('Invalid email');
}

// File 2: authController.js
function checkEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) throw new Error('Email not valid');
}

// □ AFTER – DRY
// shared/validators.js
import { z } from 'zod';
export const emailSchema = z.string().email('Invalid email');
```

2. Refactoring Strategy

2.1 AI-Guided Refactoring Workflow

- [1. PLAN] → AI analysis code, propose refactor plan
- [2. APPROVE] → Developer review plan, approve/reject
- [3. EXECUTE] → AI execute refactoring (or pair-program)
- [4. VERIFY] → AI run tests, lint, type-check
- [5. COMMIT] → AI create structured commit message

2.2 Plan Phase — AI Output

```
## Refactoring Plan: `processOrder` function
```

```
### Current State
```

- 72 lines, 5 responsibilities
- 6 parameters
- Mixed concerns: validation, data fetching, business logic, payment

```
### Target Architecture
```

```

processOrder(orderInput) → orchestrator ── validateOrderInput(orderInput) ── fetchOrderEntities(orderInput) → { order, user, product, inventory } ── calculateOrderPricing(pricingInput) → { subtotal, discount, tax, shipping, total } ── executePayment(paymentInput) → paymentResult ── updateInventory(productId, quantity) ── notifyOrderConfirmation(userId, orderConfirmation)

```

Steps

1. Extract `validateOrderInput()` – move validation (20 lines)
2. Extract `fetchOrderEntities()` – move DB calls (15 lines)
3. Create `PricingCalculator` class (25 lines)
4. Extract payment logic into `PaymentService`
5. Extract notification into `NotificationService`
6. Update `processOrder` to orchestrate (8 lines)

Risk Assessment

- Medium risk: PricingCalculator changes affect order total logic
- Low risk: Other extractions are pure moves
- Mitigation: Write characterisation tests before refactoring

2.3 Execute Phase — AI Assisted

Execute step 1: Extract validateOrderInput.

Current code (L1-L12):

```

if (!orderId) throw new Error('Missing orderId');
if (!userId) throw new Error('Missing userId');
...

```

Refactored:

```

function validateOrderInput(input) {
  const schema = z.object({
    orderId: z.string().uuid(),
    userId: z.string().uuid(),
    paymentMethod: z.enum(['card', 'gopay', 'bank_transfer']),
    shippingAddress: z.string().min(10),
    couponCode: z.string().optional(),
  });
  return schema.parse(input);
}

```

2.4 Verify Phase

```

# AI runs verification suite
npm run lint           # [ ] Pass

```

```
npx tsc --noEmit      # ☐ Pass
npm run test          # ☐ All 47 tests pass
npm run test -- --coverage # ☐ Lines: 88% (was 85%)
```

2.5 Commit Phase — AI-Generated Message

refactor(orders): extract processOrder into focused modules

- Extract validateOrderInput with zod schema
- Break up monolithic processOrder into 5 smaller functions
- Create PricingCalculator class for pricing logic
- Create PaymentService for payment processing
- Add characterisation tests for legacy behaviour
- Preserve all existing business rules

Closes #342

3. Migration Patterns

3.1 Library Upgrade (Express v4 → v5)

AI Prompt:

I need to upgrade Express from v4 to v5 in this codebase.

Current: Express v4.18.x

Target: Express v5.0.x

Breaking changes to handle:

- ``app.del()`` removed → use ``app.delete()``
- ``res.send(body)`` no longer auto-ends
- Async error handling: errors must be caught explicitly
- Path route matching changes

Generate migration guide with code diffs for each change.

3.2 Framework Migration (JavaScript → TypeScript)

AI Prompt:

Migrate this JavaScript Express API to TypeScript.

Requirements:

- Add TypeScript config (strict mode)
- Type all request/response handlers

- Create shared type definitions
- Keep existing runtime behaviour unchanged

Process:

1. First, rename all .js to .ts
2. Add type annotations
3. Create interfaces for models
4. Fix any type errors
5. Run full test suite

Generate the migration step-by-step with code examples.

3.3 Pattern Change (Callback → Async/Await)

```
// BEFORE: Callback hell
function getUserOrders(userId, callback) {
  db.query('SELECT * FROM users WHERE id = ?', [userId], (err, user) => {
    if (err) return callback(err);
    db.query('SELECT * FROM orders WHERE user_id = ?', [user.id], (err, orders)
      if (err) return callback(err);
      callback(null, orders);
    });
  });
}

// AFTER: Async/await
async function getUserOrders(userId) {
  const [user] = await db.query('SELECT * FROM users WHERE id = ?', [userId]);
  const orders = await db.query('SELECT * FROM orders WHERE user_id = ?', [user.id]);
  return orders;
}
```

4. Monolith → Service Extraction

4.1 AI-Assisted Service Boundary Detection

Prompt:

Analyze this monolithic application and propose service boundaries.

Current structure:

```
src/
  controllers/    # 15 controllers
  models/        # 20 models
```

```
services/      # 8 services (tightly coupled)
middleware/    # 5 middleware
utils/         # 12 utilities
```

Business domains:

- User management (auth, profile, settings)
- Product catalog (CRUD, categories, search)
- Order processing (cart, checkout, payment, shipping)
- Inventory management (stock, warehouse, suppliers)
- Notification (email, SMS, push)

Use Domain-Driven Design principles.

Identify:

1. Which services to extract first (highest coupling)
2. Shared kernel vs anticorruption layer
3. Event-driven boundaries vs REST boundaries
4. Data ownership per service

4.2 Extraction Strategy

Example: Extract Notification Service

```
// □ BEFORE: Tight coupling inside monolith
// orderService.js – directly calls email service
const nodemailer = require('nodemailer');

async function confirmOrder(order) {
  // ... business logic ...

  // Tight coupling to email implementation
  const transporter = nodemailer.createTransport({ ... });
  await transporter.sendMail({
    to: order.userEmail,
    subject: 'Order Confirmed',
    text: `Your order ${order.id} is confirmed.`
  });
}

// □ AFTER: Service extraction with event
// orderService.js – publishes event only
async function confirmOrder(order) {
  // ... business logic ...

  await eventBus.publish('order.confirmed', {
    orderId: order.id,
    userEmail: order.userEmail,
```

```

        total: order.total,
    });
}

// notification-service/index.js - handles event
eventBus.subscribe('order.confirmed', async (event) => {
    await emailService.send({
        to: event.userEmail,
        template: 'order-confirmation',
        data: { orderId: event.orderId, total: event.total },
    });
});
});

```

4.3 Strangler Fig Pattern

[Monolith] → [Monolith + New Service (side-by-side)] → [New Service replaces monolith]

AI Prompt:

I want to extract Notification from monolith using Strangler Fig pattern.

Phase 1: Add notification-service alongside monolith

- Route notification-related requests to new service
- Keep shared database initially

Phase 2: Migrate consumers

- Monolith services publish events instead of calling email directly
- New service listens and processes

Phase 3: Remove from monolith

- Delete notification code from monolith
- Transfer ownership of notification data

Generate implementation for Phase 1.

5. Database Migration

5.1 Schema Change with AI

Prompt:

Generate a migration plan for renaming `username` to `handle` in PostgreSQL.

Requirements:

- Zero downtime migration

- Old field still readable during transition
- Rollback script included
- Handle concurrent writes safely

Migration type: Expand-Contract (Meyer's pattern)

5.2 AI-Generated Migration Script

```
-- Phase 1: Expand – add new column
BEGIN;
ALTER TABLE users ADD COLUMN handle VARCHAR(255);
CREATE INDEX idx_users_handle ON users(handle);
-- Backfill in batches
UPDATE users SET handle = username WHERE handle IS NULL LIMIT 1000;
COMMIT;

-- Phase 2: Migrate application code to use handle
-- (deploy application update)

-- Phase 3: Contract – drop old column when safe
BEGIN;
ALTER TABLE users DROP COLUMN username;
DROP INDEX IF EXISTS idx_users_username;
COMMIT;
```

5.3 Rollback Plan

```
-- Rollback script
BEGIN;
ALTER TABLE users ADD COLUMN username VARCHAR(255);
UPDATE users SET username = handle;
ALTER TABLE users DROP COLUMN handle;
COMMIT;
```

5.4 Data Migration with Verification

```
// AI-generated data migration script
async function migrateEmailFormat() {
  const batchSize = 100;
  let migrated = 0;
  let errors = 0;

  while (true) {
    const users = await db.query(
      `SELECT id, email FROM users`
    );
```



```

        WHERE email !~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
        LIMIT $1`, [batchSize]
    );

    if (users.rows.length === 0) break;

    for (const user of users.rows) {
        try {
            const normalized = user.email.toLowerCase().trim();
            await db.query('UPDATE users SET email = $1 WHERE id = $2', [normalized,
            migrated++];
        } catch (err) {
            errors++;
            console.error(`Failed to migrate user ${user.id}:`, err);
        }
    }
}

console.log(`Migration complete: ${migrated} migrated, ${errors} errors`);
return { migrated, errors };
}

```

6. Latihan: AI Refactor Monolith ke Layered Architecture

Tugas

Kamu diberikan kode monolitik Express.js yang menangani order management. Refactor menggunakan AI assistance.

Current Code:

```

// monolith.js - 300+ lines, all in one file
const express = require('express');
const { Pool } = require('pg');
const app = express();
const pool = new Pool({ /* config */ });
const nodemailer = require('nodemailer');
const transporter = nodemailer.createTransport({ /* config */ });

// All in one: routes, validation, DB logic, email, auth
app.post('/api/orders', async (req, res) => {
    const { userId, productId, quantity, paymentMethod, couponCode } = req.body;

    // Validation inline

```

```

    if (!userId || !productId || !quantity || !paymentMethod) {
        return res.status(400).json({ error: 'Missing fields' });
    }

    // DB query inline
    const user = await pool.query('SELECT * FROM users WHERE id = $1', [userId]);
    if (user.rows.length === 0) return res.status(404).json({ error: 'User not found' });

    const product = await pool.query('SELECT * FROM products WHERE id = $1', [productId]);
    if (product.rows.length === 0) return res.status(404).json({ error: 'Product not found' });

    // Business logic inline
    const subtotal = product.rows[0].price * quantity;
    let discount = 0;
    if (couponCode) {
        const coupon = await pool.query('SELECT * FROM coupons WHERE code = $1', [couponCode]);
        if (coupon.rows.length > 0) {
            discount = subtotal * (coupon.rows[0].discount_percent / 100);
        }
    }
    const tax = (subtotal - discount) * 0.11;
    const total = subtotal - discount + tax;

    // Insert inline
    const order = await pool.query(
        `INSERT INTO orders (user_id, product_id, quantity, subtotal, discount, tax, total, payment_method)
        VALUES ($1,$2,$3,$4,$5,$6,$7,$8,'pending') RETURNING *`,
        [userId, productId, quantity, subtotal, discount, tax, total, paymentMethod]
    );

    // Email inline
    await transporter.sendMail({
        to: user.rows[0].email,
        subject: 'Order Confirmation',
        text: `Order #${order.rows[0].id} confirmed. Total: ${total}`,
    });

    res.status(201).json(order.rows[0]);
});

app.listen(3000);

```

Langkah-Langkah

1. **Deteksi smell** — Minta AI identifikasi semua code smell

2. **Buat rencana refactor** — AI usulkan arsitektur layered
3. **Ekstrak layer:**
 - routes/orderRoutes.js — routing
 - controllers/orderController.js — request handling
 - services/orderService.js — business logic
 - repositories/orderRepository.js — DB queries
 - validators/orderValidator.js — input validation
 - services/emailService.js — email sending
4. **Verifikasi** — test, lint, type-check tetap pass
5. **Commit** — AI buat commit message

Target Architecture

```
src/  
├── routes/  
│   └── orderRoutes.js  
├── controllers/  
│   └── orderController.js  
├── services/  
│   ├── orderService.js  
│   └── emailService.js  
├── repositories/  
│   └── orderRepository.js  
├── validators/  
│   └── orderValidator.js  
├── middleware/  
│   └── errorHandler.js  
└── app.js
```

Kriteria Sukses

- ☐ Semua code smell teridentifikasi (min 4 jenis)
- ☐ Refactor plan jelas dengan step-by-step
- ☐ Layered architecture terimplementasi
- ☐ Database tetap konsisten (tidak ada data loss)
- ☐ Semua endpoint berfungsi sama seperti sebelumnya
- ☐ Test suite tetap passing 100%
- ☐ Commit message terstruktur (conventional commit)

Advanced Challenge

- **Database migration:** Tambah kolom shipping_address ke tabel orders dengan zero-downtime migration
- **Service extraction:** Extract emailService menjadi microservice terpisah (simulasi dengan event bus)

- **Pattern migration:** Ganti pg client dengan Prisma ORM menggunakan AI assistance
-

Referensi

- [Martin Fowler — Refactoring](#)
- [Code Smells Catalog](#)
- [Strangler Fig Pattern](#)
- [Expand-Contract Migration Pattern](#)
- [Domain-Driven Design](#)
- [Conventional Commits](#)